

DCA-01 - Deterministic Cluster Architecture

Complete Technical Dossier - Build-Ready Specification

From Monolithic Model to Deterministic AI Ecosystem

Version: 1.2 COMPLETE DOSSIER EN
Author: Emanuel Schaaf
Date: 2026-10-06
Status: Implementable - All Schemas Included
Template: Just-in-Sequence Manufacturing (Automotive) + Triple-Modular-Redundancy TMR (Avionics) + CAN-Bus / Blackboard + LRU Principle

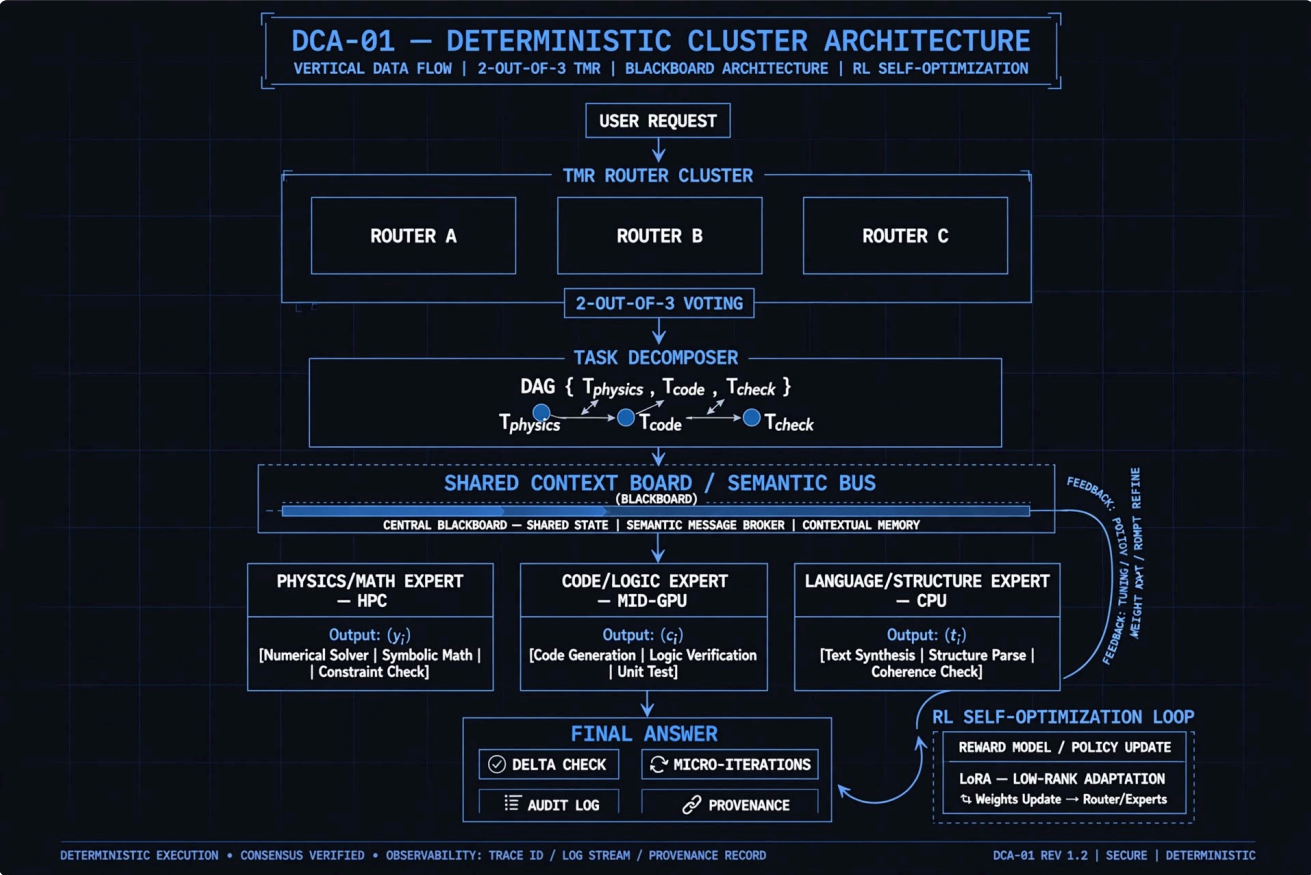


Table of Contents

1. Executive Summary and Vision
 2. Design Principles
 3. Overall Architecture and Data Flow
 4. Module 1: TMR Router Cluster - Eliminating SPOF
 5. Module 2: Task Decomposer - DAG Generation
 6. Module 3: Semantic Bus - Shared Context Board
 7. Module 4: Domain Experts - Isolated Base Models
 8. Module 5: Deterministic Fusion Layer
 9. Complete Data Models and JSON Schemas
 10. Algorithms and Pseudocode
 11. Hardware Mapping and Cost Model
 12. Self-Optimization - RL Loop and Continual Learning
 13. Observability, Logging and Metrics
 14. Security, Isolation and Signing
 15. Compliance Mapping EU AI Act
 16. Deployment, Lifecycle and Model Registry
 17. Failure Modes and Recovery
 18. Implementation Roadmap MVP to Production
 19. Glossary
 20. Appendices
-

1. Executive Summary and Vision

Goal is an AI infrastructure that fulfills four requirements simultaneously:

- **Self-optimizable:** Each module learns from its own failures without global retraining
- **Easier to train:** Local LoRA training, no global re-certification
- **More accurate and faster:** Only relevant experts active, parallel instead of serial, calibrated confidence
- **Auditable:** Every decision path logged with provenance

Instead of a monolithic giant model, a cluster of physically separated, domain-specialized base models is orchestrated via a deterministic data bus.

Core principle: Knowledge transfer does not happen through mixed weights, but through typed data exchange on a central board. Training, certification and hardware assignment are local per node. The blackbox problem is physically shattered.

For Meta Muse this means: Not one large Llama that can do everything, but an orchestrator that controls many small, highly optimized Llamas.

2. Design Principles

1. **Hard Physical Isolation:** Each expert model has its own weights, own loss, own evaluation, own hardware. No shared parameter space.
2. **Determinism over Probability:** Same input, same model hash and same seed must produce same output.
3. **TMR for Critical Paths:** Routing is triple redundant with 2-out-of-3 voting and architectural diversity.
4. **Typed Data Exchange instead of Weight Mixing:** Cross-domain transfer via blackboard pattern, not via shared network.
5. **LRU Principle:** Every unit is Line Replaceable. Exchangeable, rollback capable, without total system stop.
6. **Audit by Design:** Every decision has trace ID, provenance and signed model hash.

3. Overall Architecture and Data Flow

```
[Client Request]
|
v
[1. Input Validation + Auth + Prompt Injection Filter]
|
v
[2. TMR Router Cluster Stage 1 - 3x 100-300M parallel]
|   R_a, R_b, R_c -> Voting -> validated_intent + dag_proposal
v
[3. Task Decomposer Stage 2 - DAG Builder]
|   T_total -> {T1...Tn} with Hardware, Budget, Output Spec
v
[4. Shared Context Board / Semantic Bus - Publish/Subscribe]
|   TTL 5000ms, Schema Registry, ACL
+---<---> [Expert Physics/Math - HPC H100]
+---<---> [Expert Code/Logic - Mid GPU A6000]
+---<---> [Expert Language/Structure - CPU Cluster]
+---<---> [Expert Ethics/Logic - CPU]
|
v
[5. Fusion Layer - Delta Check + Micro-Iterations]
|   E_i(T_i) = (y_i, c_i, t_i, hash, provenance)
|   if delta > 0.25 or type_check fail -> Constraint -> Re-Run only
faulty module
v
[6. Final Answer + Provenance Graph + Audit Log + Flags]
|   Flags: CONVERGENT, NON_CONVERGENT, REVIEW_REQUIRED,
TMR_DISAGREEMENT
v
[7. Observability + RL Self-Optimization Loop -> LoRA Dataset]
```

Every request gets a `dag_id` as OpenTelemetry Trace ID. All logs reference this ID.

4. Module 1: TMR Router Cluster

4.1 Why TMR

A single router is Single Point of Failure. If it fails, everything fails. In avionics critical controls are triple redundant. Same here.

4.2 Router Specification

Three quantized routers with diversity in architecture and training data:

- Router A: Transformer 250M, trained on Intent Classification, Data Slice A
- Router B: Mamba State-Space 180M, trained on Task Graphs, Data Slice B
- Router C: Distilled 120M, trained only on Failure Cases of A and B, Data Slice C

All routers: quantized INT8, latency target < 50ms p95, Edge TPU capable.

Output Format Router:

```
{
  "router_id": "router_a_v1.2",
  "model_hash": "sha256:a1b2c3",
  "intent_label": "code_generation_with_physics",
  "confidence_r": 0.92,
  "dag_proposal": {
    "dag_hash": "sha256:d4e5f6",
    "nodes": ["T_physics", "T_code", "T_check"],
    "edges": [["T_physics", "T_code"], ["T_code", "T_check"]]
  },
  "reasoning_trace": "Request contains thermodynamics + Python,
therefore split...",
  "latency_ms": 32
}
```

4.3 Voting Protocol Complete

```
function tmr_vote(R_a, R_b, R_c):
    if hash(R_a.dag_proposal) == hash(R_b.dag_proposal):
        return majority(R_a, R_b, confidence_weighted)
    if hash(R_a.dag_proposal) == hash(R_c.dag_proposal):
        return majority(R_a, R_c)
    if hash(R_b.dag_proposal) == hash(R_c.dag_proposal):
        return majority(R_b, R_c)

    // total disagreement - critical case
    validated = max_by_confidence(R_a, R_b, R_c)
    validated.flag = "TMR_DISAGREEMENT"
    validated.flag2 = "REVIEW_REQUIRED"
    log_audit(
        event="tmr_disagreement",
        dag_id=current_dag_id,
        proposals=[R_a,R_b,R_c],
        confidence_spread=max(c)-min(c)
    )
    trigger_dataset_write("router_failures.jsonl", [R_a,R_b,R_c])
    return validated
```

Metrics Router:

- router_disagreement_rate target < 5%
- router_latency_p95 target < 50ms
- routing_precision > 95% measured against Golden Set

Loss Router: $L = 0.6 * CE(intent) + 0.2 * latency_penalty + 0.2 * disagreement_penalty$

4.4 Model Registry Router

Each router versioned, signed, with training data hash. Rollback < 30s. Registry stores eval report.

5. Module 2: Task Decomposer

5.1 Task

Decompose validated intent into executable DAG. Decomposer is itself a small LLM 1B with planner capability, not a rule set.

5.2 Complete Node Schema

```
{
  "node_id": "T_physics_01",
  "dag_id": "dag_20261006_123456",
  "domain": "thermodynamics",
  "objective": "calculate heat transfer coefficient for shell-and-tube
heat exchanger, water/water",
  "input_refs": ["user_request"],
  "target_hardware": "hpc_h100",
  "compute_budget_ms": 800,
  "timeout_ms": 1200,
  "retries": 1,
  "priority": 1,
  "output_spec": {
    "format": "json",
    "schema_ref": "schemas/physics_output_v1.json",
    "required_fields": ["Q_dot", "delta_T", "h_coefficient", "unit"],
    "determinism": {
      "temperature": 0,
      "seed": 42,
      "top_p": 1.0,
      "top_k": 1
    }
  },
  "dependencies": [],
  "blackboard_subscriptions": ["CLOSED_LOOP", "FLOW_PATTERN"]
}
```

Example DAG for "Write Python simulation for heat exchanger":

- T_physics_01: Formula derivation, no dependencies
- T_code_01: Python implementation, depends_on T_physics_01, reads its output + blackboard pattern
- T_check_01: Unit test and type check, depends_on T_code_01, T_physics_01

Decomposer also generates `dag_hash` for reproducibility.

6. Module 3: Semantic Bus - Shared Context Board

6.1 Problem and Solution

Hard isolation prevents catastrophic forgetting, but blocks accidental cross-domain transfer. Solution: Transfer on communication level like CAN-Bus in car.

Before deep inference, each module extracts abstract patterns and publishes them.

6.2 Complete Pattern Schema

```
{
  "pattern_id": "CLOSED_LOOP_01",
  "version": "1.0",
  "dag_id": "dag_20261006_123456",
  "source_module": "physics_expert",
  "source_model_hash": "sha256:physics_v2.1",
  "timestamp": "2026-10-06T12:00:00Z",
  "ttl_ms": 5000,
  "pattern_type": "topological",
  "abstraction": {
    "name": "closed loop system",
    "description": "Energy conservation, feedback required",
    "properties": {
      "conservation": true,
      "feedback_loop": true,
      "steady_state_required": true,
      "suggested_code_pattern": "while_loop_with_convergence"
    }
  },
  "vector_embedding": [0.12, 0.88, 0.34],
  "access_control": {
    "write": ["physics_expert"],
    "read": ["code_expert", "logic_expert", "language_expert"]
  },
  "consumers_ack": [],
  "audit": true
}
```

Example transfer: Physics detects closed loop. Code reads `suggested_code_pattern: while_loop_with_convergence` and chooses `while abs(delta) > eps` instead of `for`. No physics weights in code model needed.

6.3 Bus API Contract

```
publish(pattern: Pattern) -> bool, validated against Schema Registry,  
ACL check  
subscribe(filter: domain|pattern_type) -> stream<Pattern>  
ack(pattern_id, module_id) -> bool  
get_patterns(dag_id) -> list<Pattern>  
get_audit_log(dag_id) -> list<Pattern>
```

Bus itself redundant: two lines like CAN-Bus with own health check. Rate limit against poisoning. Schema validation.

Rules:

- Write only in own namespace
- Read globally but only typed fields
- No direct module-to-module call
- TTL and versioning, automatic GC after TTL
- Every publish logged with dag_id

7. Module 4: Domain Experts

7.1 Isolation

Each expert model: own weights, own loss function, own evaluation, own hardware. No shared encoder.

- Physics/Math Expert: Loss = numerical stability + unit correctness + formula consistency. Trained on textbooks, papers, simulations.
- Code/Logic Expert: Loss = syntax validity + unit test pass rate + type correctness. Trained on code + tests.
- Language/Structure Expert: Loss = semantic coherence + format fidelity Markdown/JSON.
- Ethics/Logic Expert: Loss = rule compliance + consistency.

7.2 Expert Output Format

$$E_i(T_i) = (y_i, c_i, t_i, model_{hash}, provenance)$$

```
{
  "node_id": "T_physics_01",
  "module_id": "physics_expert_v2.1",
  "model_hash": "sha256:physics_v2.1",
  "y_i": {
    "Q_dot": 15000,
    "delta_T": 25,
    "h_coefficient": 1200,
    "unit": "W/m2K",
    "formula": "Q_dot = h * A * delta_T"
  },
  "c_i": 0.94,
  "c_i_calibrated": 0.91,
  "ece_score": 0.03,
  "t_i_ms": 620,
  "provenance": {
    "used_patterns": ["CLOSED_LOOP_01"],
    "input_refs": ["user_request"],
    "blackboard_reads": 1
  }
}
```

Confidence calibration: Platt Scaling on validation set, target ECE < 0.05. Without calibration delta check is worthless.

7.3 Determinism per Expert

- Temperature 0, top_p 1.0, top_k 1, fixed seed
- Reproducible CUDA kernels, deterministic algorithms
- Output validated against `output_spec.schema_ref` before forwarding. On schema violation: retry with constraint.

8. Module 5: Deterministic Fusion Layer

8.1 Task

Fusion must not concatenate. It must resolve contradictions like final assembly in automotive factory. Enforce physical and logical compatibility.

8.2 Conflict Taxonomy

- TYPE_MISMATCH: int vs float64
- UNIT_MISMATCH: W vs kW, Celsius vs Kelvin
- CONFIDENCE_GAP: $|c_{\text{physics}} - c_{\text{code}}| > 0.25$
- LOGIC_CONTRADICTION: Formula says A, code implements B
- FORMAT_VIOLATION: Output not valid against schema

8.3 Fusion Algorithm Complete

```
function fuse(expert_outputs: list<E_i>, dag_id):
    delta_conf = max(c_i) - min(c_i)
    type_ok = validate_all_against_schema(expert_outputs)
    unit_ok = validate_units(expert_outputs)
    logic_ok = check_logic_consistency(expert_outputs)

    if delta_conf > 0.25 or !type_ok or !unit_ok or !logic_ok:
        constraint = {
            type: detect_conflict_type(),
            detail: "Variable typing in code violates physical floating
precision: float64 required, int found. Pattern CLOSED_LOOP_01 requires
while with convergence.",
            expected: load_schema("physics_output_v1.json"),
            actual: expert_outputs.code.y_i,
            delta: delta_conf,
            dag_id: dag_id,
            failing_module: "code_expert"
        }
        log_audit(event="fusion_constraint", constraint=constraint,
dag_id=dag_id)

        if iteration_count < MAX_ITER=3:
            trigger_rerun(module=constraint.failing_module,
extra_input=constraint, budget_increase=20%)
            iteration_count++
            return fuse(new_outputs)
        else:
            return {
                answer: best_effort_merge(expert_outputs),
                flag: "NON_CONVERGENT",
                provenance: collect_provenance(expert_outputs),
                audit: get_audit_log(dag_id),
                constraints_tried: all_constraints
            }
    else:
        return
```

```

return {
    answer: merge_with_template(expert_outputs),
    flag: "CONVERGENT",
    provenance: collect_provenance(expert_outputs),
    audit: get_audit_log(dag_id),
    confidence_final: mean(c_i_calibrated)
}

```

Merger: Template based, deterministic, no free generation. Temperature 0, fixed seed.

8.4 Final Answer Format

```

{
  "dag_id": "dag_20261006_123456",
  "final_answer": "... synthesized answer with formulas and code ...",
  "flag": "CONVERGENT",
  "confidence_final": 0.92,
  "provenance_graph": {
    "router_votes": ["router_a", "router_b"],
    "dag_hash": "sha256:d4e5",
    "experts_used": [
      {"module": "physics_expert_v2.1", "hash": "sha256:...", "c": 0.91},
      {"module": "code_expert_v3.0", "hash": "sha256:...", "c": 0.93}
    ],
    "blackboard_patterns": ["CLOSED_LOOP_01"],
    "fusion_iterations": 1,
    "model_hashes": ["sha256:..."]
  },
  "audit_log_ref": "s3://audit/dag_20261006_123456.jsonl",
  "generated_at": "2026-10-06T12:01:00Z"
}

```

9. Complete Data Models and JSON Schemas

Directory /schemas/ :

- router_output_v1.json - Router Output
- dag_node_v1.json - DAG Node Spec
- blackboard_pattern_v1.json - Blackboard Pattern
- expert_output_v1.json - Expert Output E_i
- physics_output_v1.json - Domain specific
- code_output_v1.json
- fusion_constraint_v1.json
- final_answer_v1.json
- audit_event_v1.json

All schemas versioned, in Schema Registry, validated before each publish.

Example expert_output_v1.json :

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "required":
  ["node_id", "module_id", "model_hash", "y_i", "c_i", "c_i_calibrated", "t_i_ms", "provenance"],
  "properties": {
    "node_id": {"type": "string"},
    "module_id": {"type": "string"},
    "model_hash": {"type": "string", "pattern": "^sha256:"},
    "y_i": {"type": "object"},
    "c_i": {"type": "number", "minimum": 0, "maximum": 1},
    "c_i_calibrated": {"type": "number"},
    "t_i_ms": {"type": "integer"},
    "provenance": {"type": "object"}
  }
}
```

10. Algorithms and Pseudocode

See Modules 4,5,8. Additionally:

- **DAG Builder Algorithm:** Topological sorting, cycle detection, budget distribution by priority
- **ECE Calibration:** Binning method, Platt Scaling training monthly
- **LoRA Training:** Rank 8-16, only on failure dataset, early stopping on ECE increase

11. Hardware Mapping and Cost Model

Module	Hardware	vCPU	RAM	GPU	Cost per 1k Requests	Scaling
TMR Router	Edge TPU / NPU	2	4GB	NPU	0.02€	Horizontal per Request
Physics Expert	H100 80GB	8	32GB	1x H100	0.45€	Queue + Autoscale 0-10
Code Expert	A6000 48GB	4	16GB	1x A6000	0.12€	Parallel
Language Expert	CPU / T4	4	8GB	0-1x T4	0.03€	Edge
Fusion Layer	A10 24GB	4	16GB	1x A10	0.05€	Single
Blackboard / Bus	CPU	2	8GB	0	0.01€	Redundant 2x

Advantage: Simple text request costs only Router + Language = 0.05€ instead of 0.62€ for full pipeline. Power and compute only flow where router allows.

12. Self-Optimization - RL Loop and Continual Learning

12.1 Logging

Every DAG path logged: dag_id, router votes, decomposer output, blackboard patterns, E_i with c_i, t_i, fusion constraints, iterations.

Storage: JSONL in S3, OpenTelemetry Trace.

12.2 Trigger for Training

- When c_i drops at same t_i over 100 requests -> drift
- When micro-iterations for module C > 2 on average -> hallucination
- When TMR disagreement > 10% -> router outdated
- When type_check fail rate > 2% -> schema or model outdated

12.3 Training Flow

1. Failure log -> dataset_module_C_failures_v{n}.jsonl with constraint as label and correct output as target
2. Shadow training isolated: LoRA Rank 8-16, only module C, no global training
3. Eval against module own golden benchmark + ECE check
4. Canary deployment: 5% traffic on new LoRA version, monitoring 1h
5. On success: Model Registry update, old hash archived, new hash active, audit entry
6. Rest stays fully certified, no global re-certification
7. On failure: Auto rollback to last good hash, alert, dataset review

This ends re-certification hell. Update is local, not global. LRU principle.

13. Observability, Logging and Metrics

Metrics as Prometheus:

- `dca_router_disagreement_rate`
- `dca_router_latency_p95_ms`
- `dca_dag_decomposition_latency_ms`
- `dca_blackboard_publish_rate_per_dag`
- `dca_blackboard_ttl_expired_total`
- `dca_fusion_convergence_rate`
- `dca_fusion_iterations_histogram`
- `dca_expert_ece_score{module="physics"}`
- `dca_expert_latency_p95_ms{module="code"}`
- `dca_expert_failure_rate{module="code", type="type_mismatch"}`

Logging: JSONL with fields `timestamp`, `dag_id`, `event_type`, `module_id`, `model_hash`, `latency`, `confidence`, `constraint`.

Dashboard: Grafana with DAG Trace View.

14. Security, Isolation and Signing

- Each expert unit in own sandbox, Docker + gVisor, no network sharing except bus port
- Models signed with Cosign, hash verified before inference. No unsigned model allowed to start.
- Input validation before router: Prompt injection filter, PII filter, max token check
- Blackboard: Schema validation, ACL, rate limit 100 publishes per dag_id against poisoning
- Fusion can hard shutdown experts at $c_i < 0.2$ or repeated fail
- Audit log immutable, WORM storage S3 Object Lock
- Secrets: Model registry credentials via Vault, no hardcoding

15. Compliance Mapping EU AI Act

- Art. 9 Risk Management: Modular risk analysis per node, failure modes documented
- Art. 10 Data Governance: Training data hashes logged, failure datasets versioned
- Art. 11 Technical Documentation: This dossier + provenance graph + model hashes + schemas
- Art. 12 Logging: Complete audit log per DAG, immutable
- Art. 13 Transparency: Final answer contains flag CONVERGENT/NON_CONVERGENT and confidence
- Art. 14 Human Oversight: REVIEW_REQUIRED and NON_CONVERGENT go to Human-in-the-Loop queue
- Art. 15 Accuracy and Robustness: ECE monitoring, canary, rollback

16. Deployment, Lifecycle and Model Registry

- **Model Registry:** MLflow or HF Hub Enterprise. Stores model hash, training data hash, eval report, signature, LoRA base model.
- **CI/CD:** GitHub Actions: Lint schemas, unit tests per module, golden benchmark eval, sign, push registry.
- **Deployment:** Kubernetes. Each module type own deployment with HPA. Blue/Green per module. Rollback < 30s.
- **Versioning:** SemVer per module, e.g. `physics_expert v2.1.3`. DAG schema versioning separately `dag_schema v1.0`. Compatibility matrix.
- **Edge:** Router + language locally on device, HPC modules on-demand in cloud via mTLS.

17. Failure Modes and Recovery - Complete

Failure	Detection	Recovery	Audit
TMR total disagreement	No majority hash	Flag REVIEW_REQUIRED, Human Review, Router Retraining Dataset	tmr_disagreement
Blackboard TTL expired	Pattern not acked in TTL	Re-publish trigger, alert if > 3x	ttl_expired
Fusion deadlock > 3 iter	MAX_ITER reached	NON_CONVERGENT + best effort + all constraints	fusion_deadlock
Expert timeout	$t_i > \text{timeout_ms}$	Retry with +20% budget, then fallback module or degraded mode	expert_timeout
LoRA drift ECE rises	$\text{ECE} > 0.08$ after training	Auto-rollback to last good hash, alert, dataset review	lora_drift
Schema violation	JSON schema validation fail	Retry with constraint "Schema violated field X", then flag	schema_violation
Model hash mismatch	Signature invalid	Pod does not start, alert, registry check	security_violation

18. Implementation Roadmap MVP to Production

Phase 0 - MVP 2 weeks:

- 1 Router instead of TMR, decomposer hardcoded for 3 tasks
- 2 Experts: Code + Language, blackboard as in-memory dict
- Fusion simple: Merge without delta check
- Goal: End-to-end DAG runs

Phase 1 - Determinism 4 weeks:

- JSON Schemas + validation
- Confidence calibration + ECE measurement
- Fusion with delta check and 1 retry
- Audit log JSONL + dag_id

Phase 2 - TMR + Bus 6 weeks:

- 3 Routers with voting, architectural diversity
- Blackboard with Redis + Schema Registry + ACL
- Hardware mapping + budgeting
- Prometheus metrics + Grafana

Phase 3 - Self-Optimization 8 weeks:

- Model registry + signing
- LoRA training pipeline + canary + rollback
- Failure modes implemented
- EU AI Act doc and human-in-the-loop queue

Phase 4 - Production:

- Autoscaling HPA, WORM audit storage, edge deployment, cost dashboard

19. Glossary

- **DCA-01:** Deterministic Cluster Architecture Version 01
- **TMR:** Triple Modular Redundancy
- **DAG:** Directed Acyclic Graph
- **LRU:** Line Replaceable Unit
- **ECE:** Expected Calibration Error
- **LoRA:** Low-Rank Adaptation
- **Provenance:** Origin proof which models and patterns were used
- **Blackboard:** Central shared context board
- **Deterministic:** Same input + same model hash + same seed = same output

20. Appendices

Appendix A: Repo Directory Structure

```
/dca-01/  
  /schemas/  
    router_output_v1.json  
    dag_node_v1.json  
    blackboard_pattern_v1.json  
    expert_output_v1.json  
    physics_output_v1.json  
    code_output_v1.json  
    fusion_constraint_v1.json  
    final_answer_v1.json  
    audit_event_v1.json  
  /src/  
    router/  
    decomposer/  
    blackboard/  
    experts/  
    fusion/  
    observability/  
  README.md  
  docker-compose.yml
```

Appendix B: Mermaid Diagrams

```
graph TD  
  A[User Request] --> B[TMR Router]  
  B -->|2-out-of-3 Voting| C[Decomposer DAG]  
  C --> D[Blackboard Bus]  
  D --> E[Physics Expert]  
  D --> F[Code Expert]  
  D --> G[Language Expert]  
  E --> H[Fusion Layer]  
  F --> H  
  G --> H  
  H -->|Delta Check| I{Convergent?}  
  I -->|Yes| J[Final Answer CONVERGENT]  
  I -->|No <3 Iter| K[Constraint + Re-Run]  
  K --> D  
  I -->|No >=3| L[NON_CONVERGENT + Audit]
```

Appendix C: Eval Benchmarks per Expert

- Physics: 500 questions thermodynamics with unit check, target accuracy > 92%, ECE < 0.05
 - Code: HumanEval + own unit tests, target pass rate > 95%
 - Router: Golden set 1000 intents, target precision > 95%, disagreement < 5%
-

End of Dossier v1.2 COMPLETE EN

This specification is complete. It contains all contracts, schemas, algorithms, failure handling, security, compliance and lifecycle to implement DCA-01 without further questions. From idea to build-ready infrastructure.

Author: Emanuel Schaaf

License: Open Spec for industrial implementation